



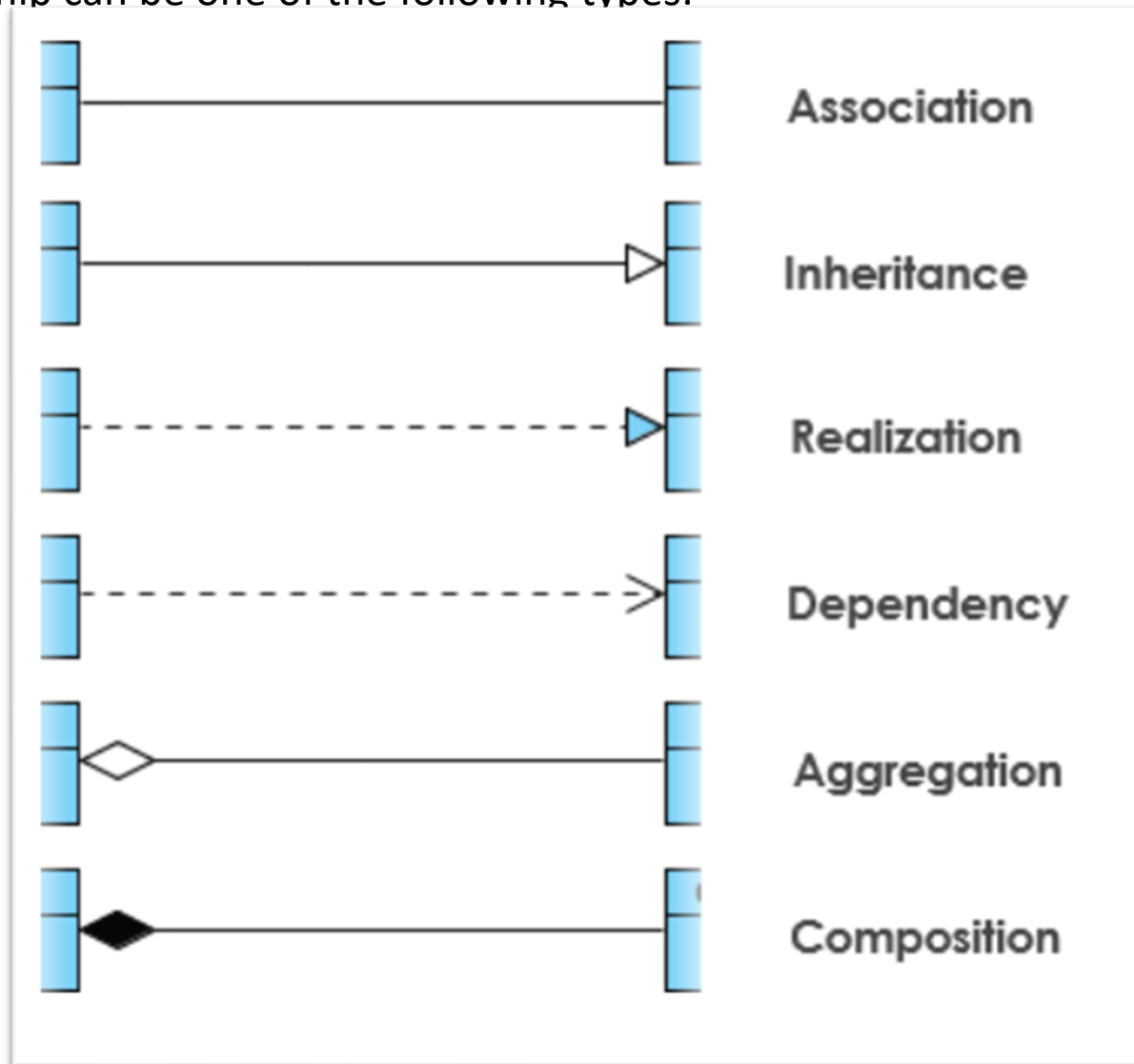
PASSION *for* TECHNOLOGY

30.05.2026 Czapiewski Paweł
POJ Ćwiczenia 06



UML CLASS NOTATION

A relationship can be one of the following types:

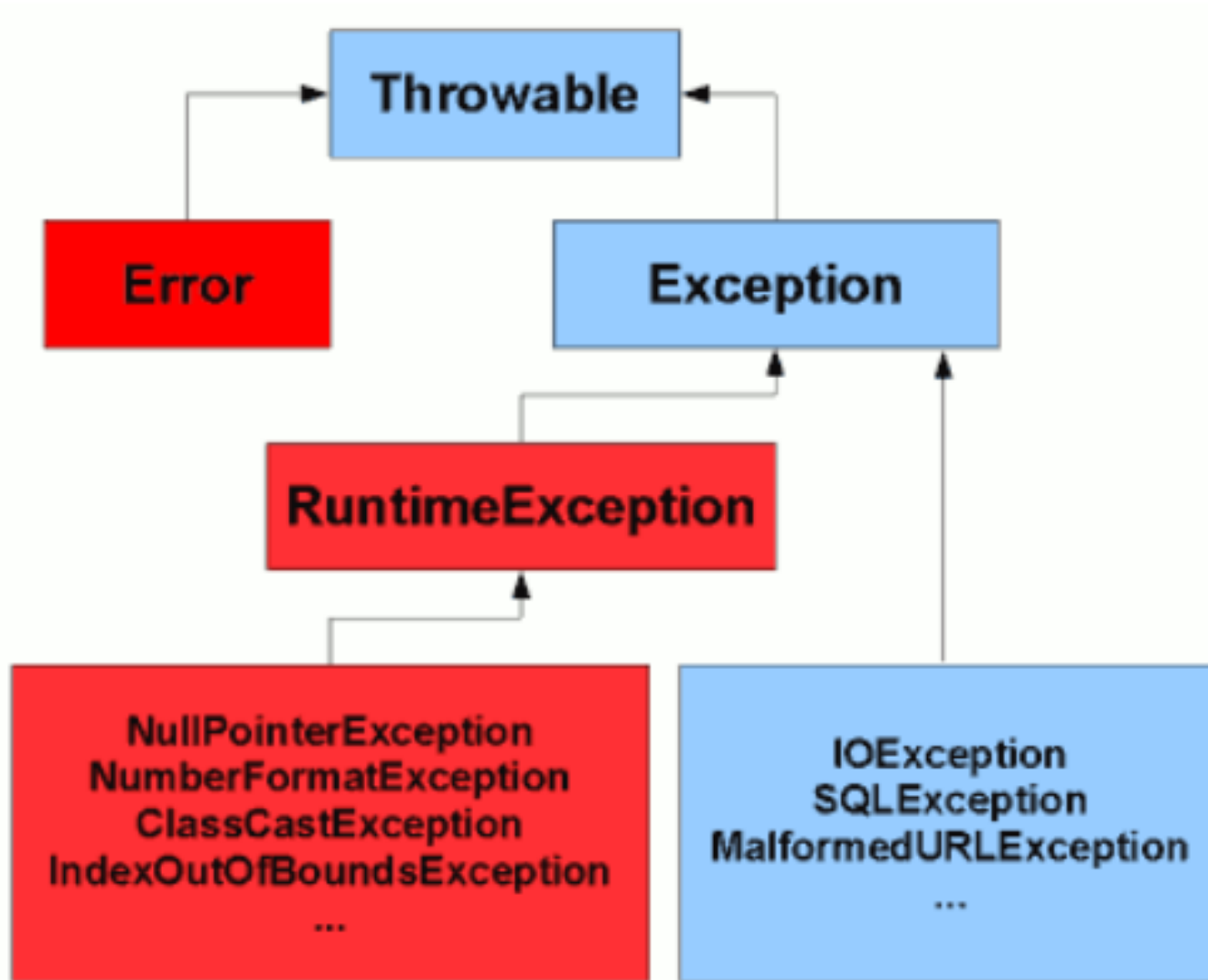


Source: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>



Source: <https://www.improgrammer.net/java-programming-jokes-best-programmer-humor/>

EXCPETION HIERARCHY



EXCEPTION HANDLING

```
try
{
    // statements to try
}
catch(Exception e)
{
    // actions that occur if exception was thrown
}
finally
{
    // actions that occur whether catch block executed or not
}
```

EXCEPTION PSEUDOCODE

```
call methodA()
if methodA() worked
{
    call methodB()
    if methodB() worked
    {
        call methodC()
        if methodC() worked
            everything's okay, so display finalResult
        else
            set errorCode to 'C'
    }
    else
        set errorCode to 'B'
}
else
    set errorCode to 'A'
```

EXCEPTION PSEUDOCODE

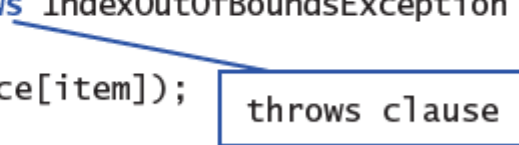
```
call methodA()
if methodA() worked
{
    call methodB()
    if methodB() worked
    {
        call methodC()
        if methodC() worked
            everything's okay, so display finalResult
        else
            set errorCode to 'C'
    }
    else
        set errorCode to 'B'
}
else
    set errorCode to 'A'
```

```
try
{
    call methodA() and maybe throw an exception
    call methodB() and maybe throw an exception
    call methodC() and maybe throw an exception
    everything's okay, so display finalResult
}
catch(methodA()'s error)
{
    set errorCode to "A"
}
catch(methodB()'s error)
{
    set errorCode to "B"
}
catch(methodC()'s error)
{
    set errorCode to "C"
}
```


EXCEPTION HANDLING

If a method throws an exception that it will not catch but that a different method will catch, you must create a throws **clause** by using the keyword throws followed by an Exception type in the method header. This practice is known as **exception specification**.

```
public class Pricelist
{
    private static final double[] prices = {15.99, 27.88, 34.56, 45.89};
    public static void displayPrice(int item) throws IndexOutOfBoundsException
    {
        System.out.println("The price is $" + price[item]);
    }
}
```



EXCEPTION HANDLING

```
import java.util.*;
public class PriceListApplication1
{
    public static void main(String[] args)
    {
        int item;
        Scanner input = new Scanner(System.in);
        System.out.print("Enter item number >> ");
        item = input.nextInt();
        try
        {
            PriceList.displayPrice(item);
        }
        catch(IndexOutOfBoundsException e)
        {
            System.out.println("Price is $0");
        }
    }
}
```

DOG-BIRD



Source: https://www.huffpost.com/entry/birds-with-dog-heads-dirds_n_4324833



Kyle Brownlee

@_kylebrownlee



how are unicorns fake but giraffes are real like what's more believable a horse with a horn or a leopard-moose-camel with a 40 foot neck

♡ 489K 3:40 PM - Jul 14, 2018

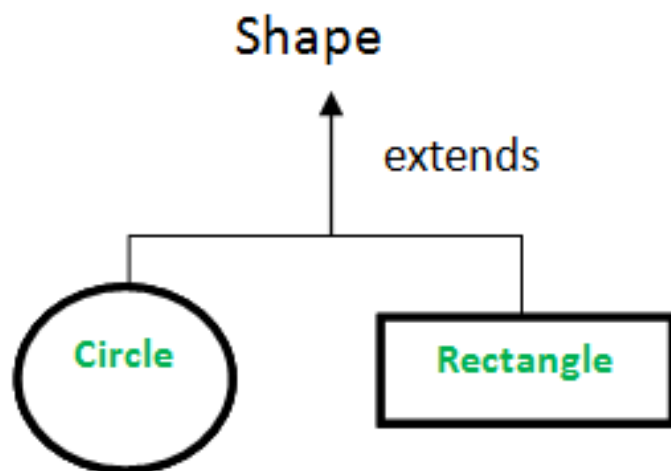


💬 138K people are talking about this

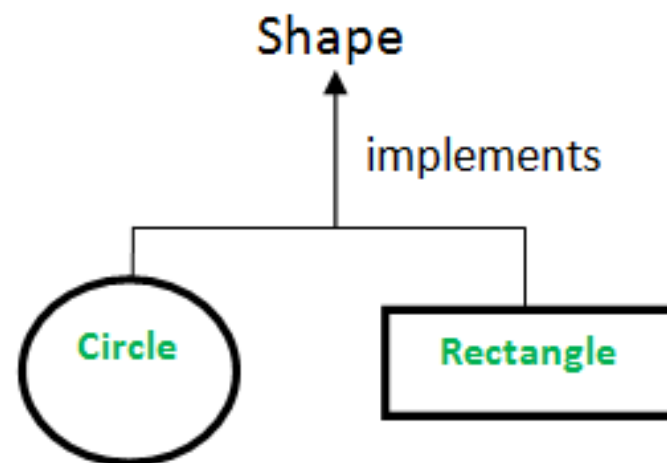


INTERFACES

Abstract Class



Interface



OOPs interface vs abstract class

Interface	Abstract class
Interface support multiple implementations.	Abstract class does not support multiple inheritance.
Interface does not contain Data Member	Abstract class contains Data Member
Interface does not contain Constructors	Abstract class contains Constructors
An interface Contains only incomplete member (signature of member)	An abstract class Contains both incomplete (abstract) and complete member
An interface cannot have access modifiers by default everything is assumed as public	An abstract class can contain access modifiers for the subs, functions, properties
Member of interface can not be Static	Only Complete Member of abstract class can be Static

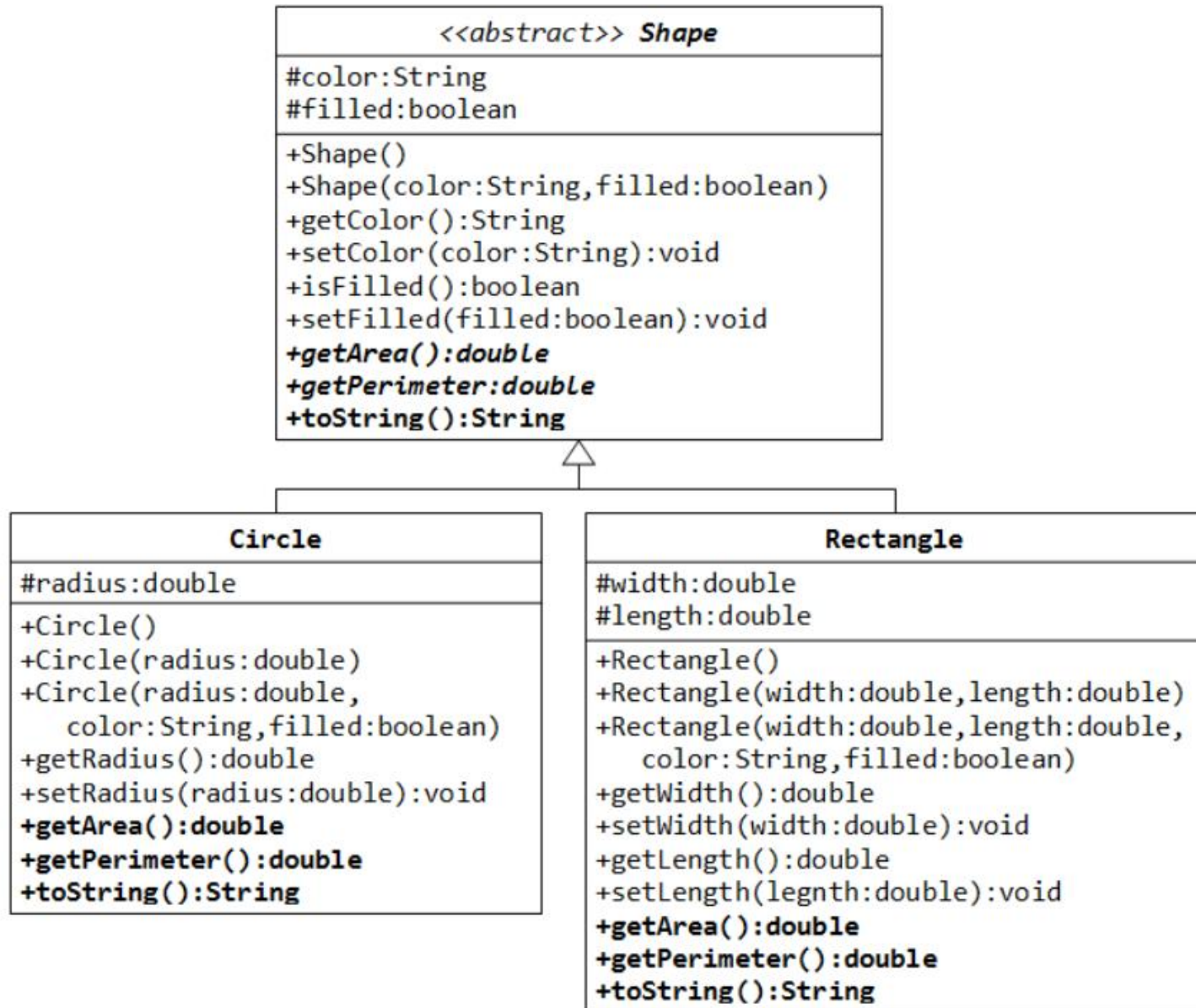
INTERFACES VS ABSTRACT CLASS

Abstract class vs Interface

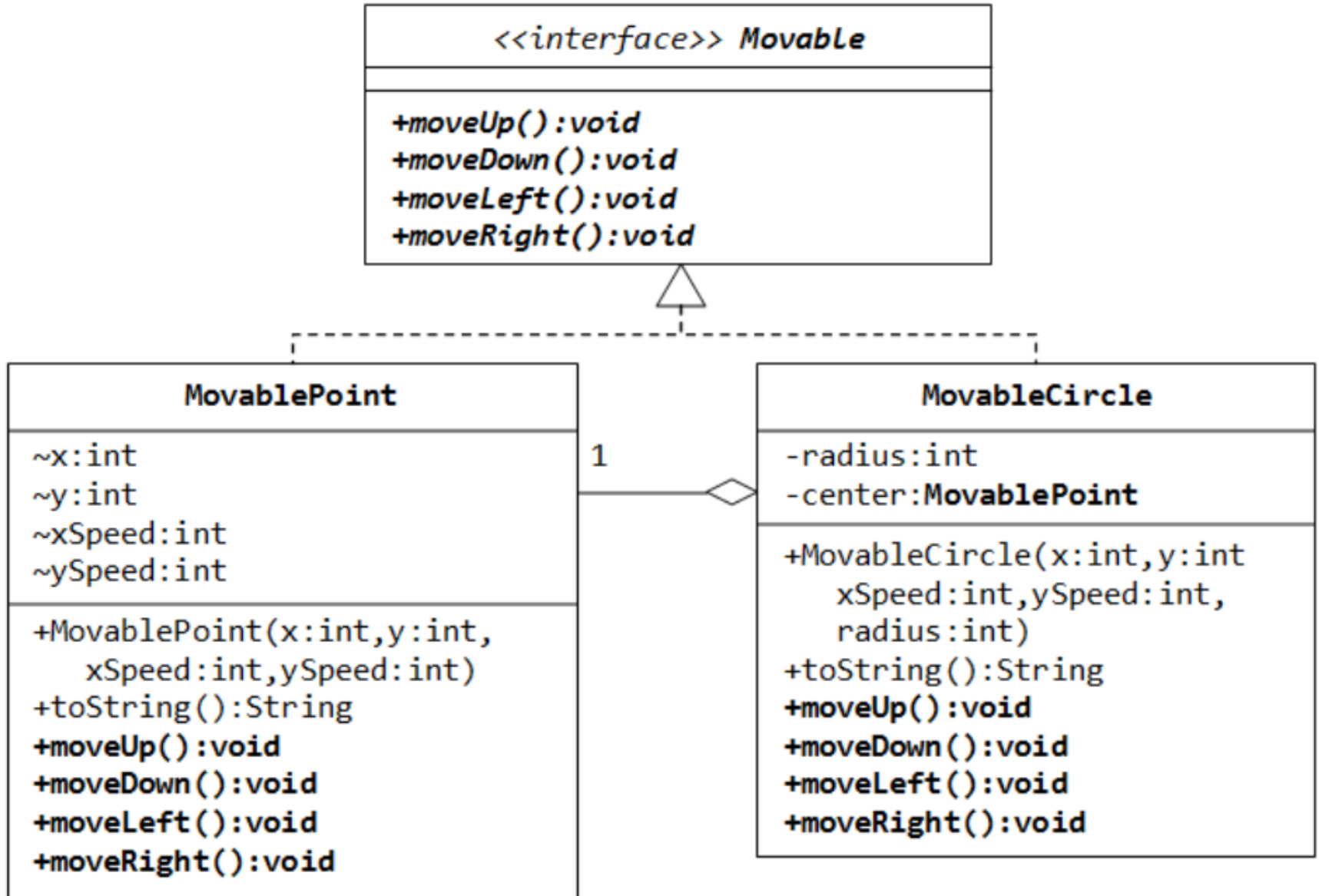
- 1.Type of methods:** Interface can have only abstract methods. Abstract class can have abstract and non-abstract methods. From Java 8, it can have default and static methods also.
- 2.Final Variables:** Variables declared in a Java interface are by default final. An abstract class may contain non-final variables.
- 3.Type of variables:** Abstract class can have final, non-final, static and non-static variables. Interface has only static and final variables.
- 4.Implementation:** Abstract class can provide the implementation of interface. Interface can't provide the implementation of abstract class.
- 5.Inheritance vs Abstraction:** A Java interface can be implemented using keyword "implements" and abstract class can be extended using keyword "extends".
- 6.Multiple implementation:** An interface can extend another Java interface only, an abstract class can extend another Java class and implement multiple Java interfaces.
- 7.Accessibility of Data Members:** Members of a Java interface are public by default. A Java abstract class can have class members like private, protected, etc.

Source: <https://www.geeksforgeeks.org/difference-between-abstract-class-and-interface-in-java/>

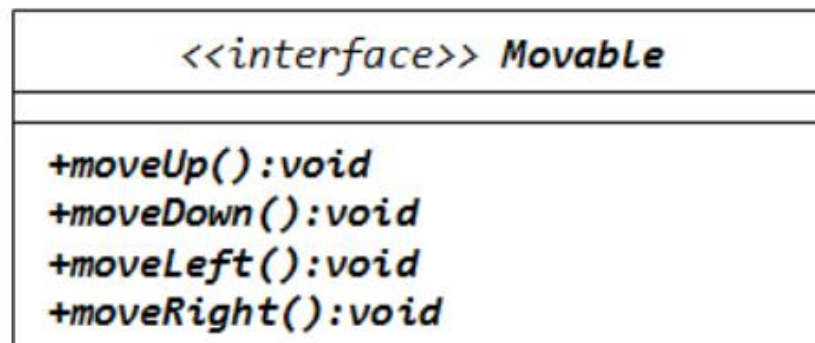
ABSTRACT CLASS



INTERFACES



INTERFACES



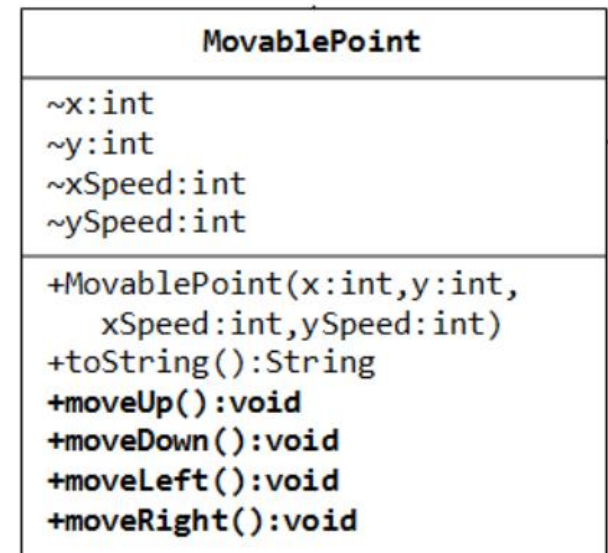
```
public interface Movable { // saved as "Movable.java"
    public void moveUp();
    .....
}
```

INTERFACES

```
public class MovablePoint implements Movable { // saved as "MovablePoint.java"
    // instance variables
    int x, y, xSpeed, ySpeed;    // package access

    // Constructor
    public MovablePoint(int x, int y, int xSpeed, int ySpeed) {
        this.x = x;
        .....
    }
    .....

    // Implement abstract methods declared in the interface Movable
    @Override
    public void moveUp() {
        y -= ySpeed;    // y-axis pointing down for 2D graphics
    }
    .....
}
```



INTERFACES

```
public class MovableCircle implements Movable { // saved as "MovableCircle.java"
    // instance variables
    private MovablePoint center;    // can use center.x, center.y directly
                                    // because they are package accessible

    private int radius;

    // Constructor
    public MovableCircle(int x, int y, int xSpeed, int ySpeed, int radius) {
        // Call the MovablePoint's constructor to allocate the center instance.
        center = new MovablePoint(x, y, xSpeed, ySpeed);
        .....
    }
    .....

    // Implement abstract methods declared in the interface
    @Override
    public void moveUp() {
        center.y -= center.ySpeed;
    }
    .....
}
```

MovableCircle
-radius:int -center:MovablePoint
+MovableCircle(x:int,y:int xSpeed:int,ySpeed:int, radius:int) +toString():String +moveUp():void +moveDown():void +moveLeft():void +moveRight():void



**A USER
INTERFACE
IS LIKE A JOKE.
IF YOU HAVE TO
EXPLAIN IT, IT'S
NOT THAT
GOOD.**

Source: <https://twitter.com/pauljadam/status/555118744351952896>

COMPARABLE INTERFACE

public int **compareTo**(Object obj): It is used to compare the current object with the specified object. It returns:

- ❑ positive integer, if the current object is greater than the specified object.
- ❑ negative integer, if the current object is less than the specified object.
- ❑ zero, if the current object is equal to the specified object.

COMPARABLE INTERFACE

```
class Student implements Comparable<Student>{  
    int rollno;  
    String name;  
    int age;  
    Student(int rollno,String name,int age){  
        this.rollno=rollno;  
        this.name=name;  
        this.age=age;  
    }  
  
    public int compareTo(Student st){  
        if(age==st.age)  
            return 0;  
        else if(age>st.age)  
            return 1;  
        else  
            return -1;  
    }  
}
```


COMPARABLE INTERFACE

```
import java.util.*;

public class TestSort1{
    public static void main(String args[]){
        ArrayList<Student> al=new ArrayList<Student>();
        al.add(new Student(101,"Vijay",23));
        al.add(new Student(106,"Ajay",27));
        al.add(new Student(105,"Jai",21));

        Collections.sort(al);
        for(Student st:al){
            System.out.println(st.rollno+" "+st.name+" "+st.age);
        }
    }
}
```

```
105 Jai 21
101 Vijay 23
106 Ajay 27
```

COMPARABLE INTERFACE

```
import java.util.Comparator;

public class Employee implements Comparable<Employee> {

    private int id;
    private String name;
    private int age;
    private long salary;

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public long getSalary() {
        return salary;
    }

    public Employee(int id, String name, int age, int salary) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.salary = salary;
    }

    // (...)
}
```

COMPARABLE INTERFACE

```
//(...)
@Override
public int compareTo(Employee emp) {
    //let's sort the employee based on an id in ascending order
    //returns a negative integer, zero, or a positive integer as this employee id
    //is less than, equal to, or greater than the specified object.
    return (this.id - emp.id);
}

/**
 * Comparator to sort employees list or array in order of Salary
 */
public static Comparator<Employee> SalaryComparator = new Comparator<Employee>() {

    @Override
    public int compare(Employee e1, Employee e2) {
        return (int) (e1.getSalary() - e2.getSalary());
    }
};

/**
 * Comparator to sort employees list or array in order of Name
 */
public static Comparator<Employee> NameComparator = new Comparator<Employee>() {

    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.getName().compareTo(e2.getName());
    }
};
}
```

COMPARABLE INTERFACE

```
import java.util.Arrays;

public class JavaObjectSorting {

    /**
     * This class shows how to sort custom objects array/list
     * implementing Comparable and Comparator interfaces
     * @param args
     */
    public static void main(String[] args) {

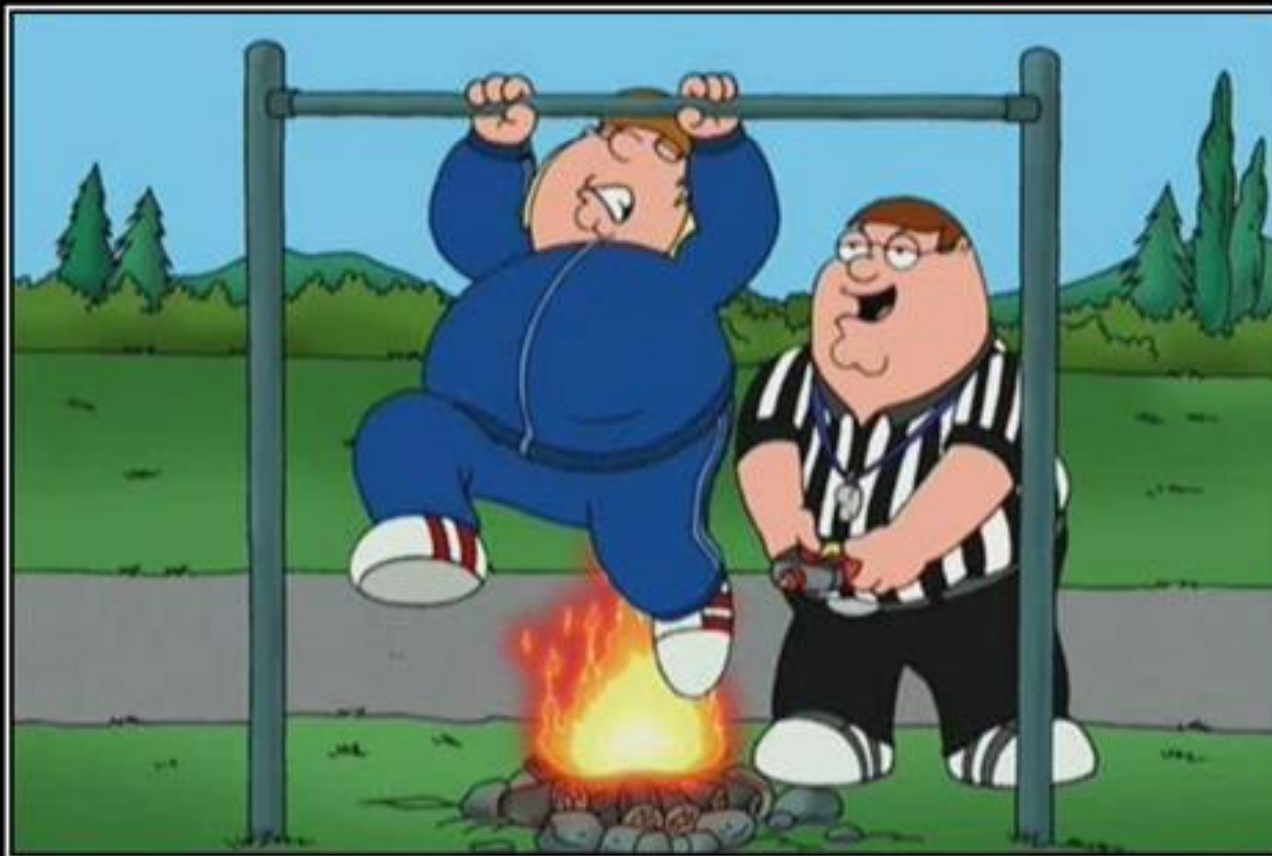
        //sorting custom object array
        Employee[] empArr = new Employee[4];
        empArr[0] = new Employee(10, "Mikey", 25, 10000);
        empArr[1] = new Employee(20, "Arun", 29, 20000);
        empArr[2] = new Employee(5, "Lisa", 35, 5000);
        empArr[3] = new Employee(1, "Pankaj", 32, 50000);

        //sorting employees array using Comparable interface implementation
        Arrays.sort(empArr);
        System.out.println("Default Sorting of Employees list:\n"+Arrays.toString(empArr));

        //sort employees array using Comparator by Salary
        Arrays.sort(empArr, Employee.SalaryComparator);
        System.out.println("Employees list sorted by Salary:\n"+Arrays.toString(empArr));

        //sort employees array using Comparator by Name
        Arrays.sort(empArr, Employee.NameComparator);
        System.out.println("Employees list sorted by Name:\n"+Arrays.toString(empArr));

    }
}
```



EXERCISE

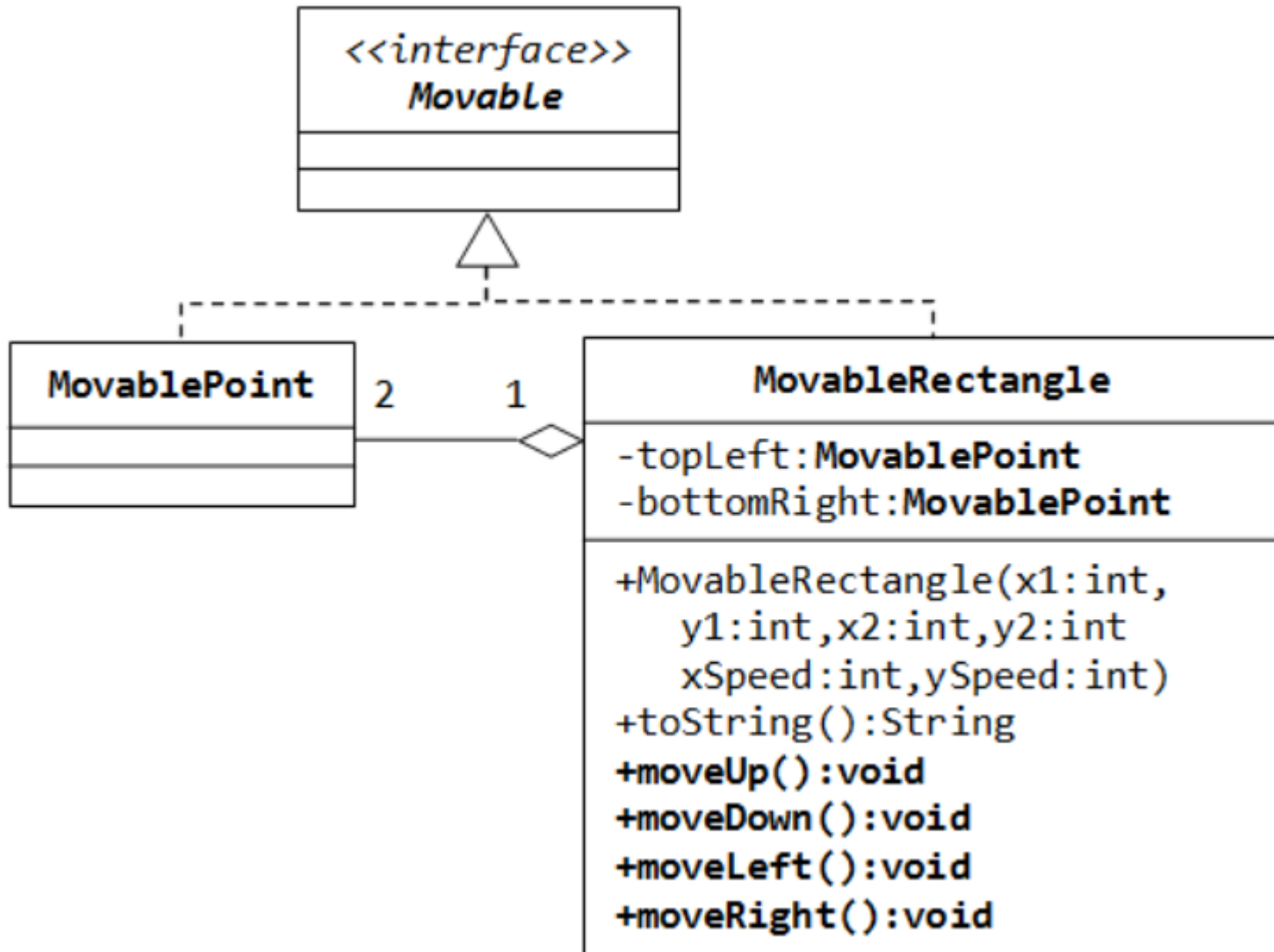
Feel the burn!!

\o/ MotivatedPhotos.com

Source: <http://www.dumpaday.com/funny-pictures/funny-fitness-pictures-34-pics/attachment/funny-fitness-pictures-feel-the-burn/>

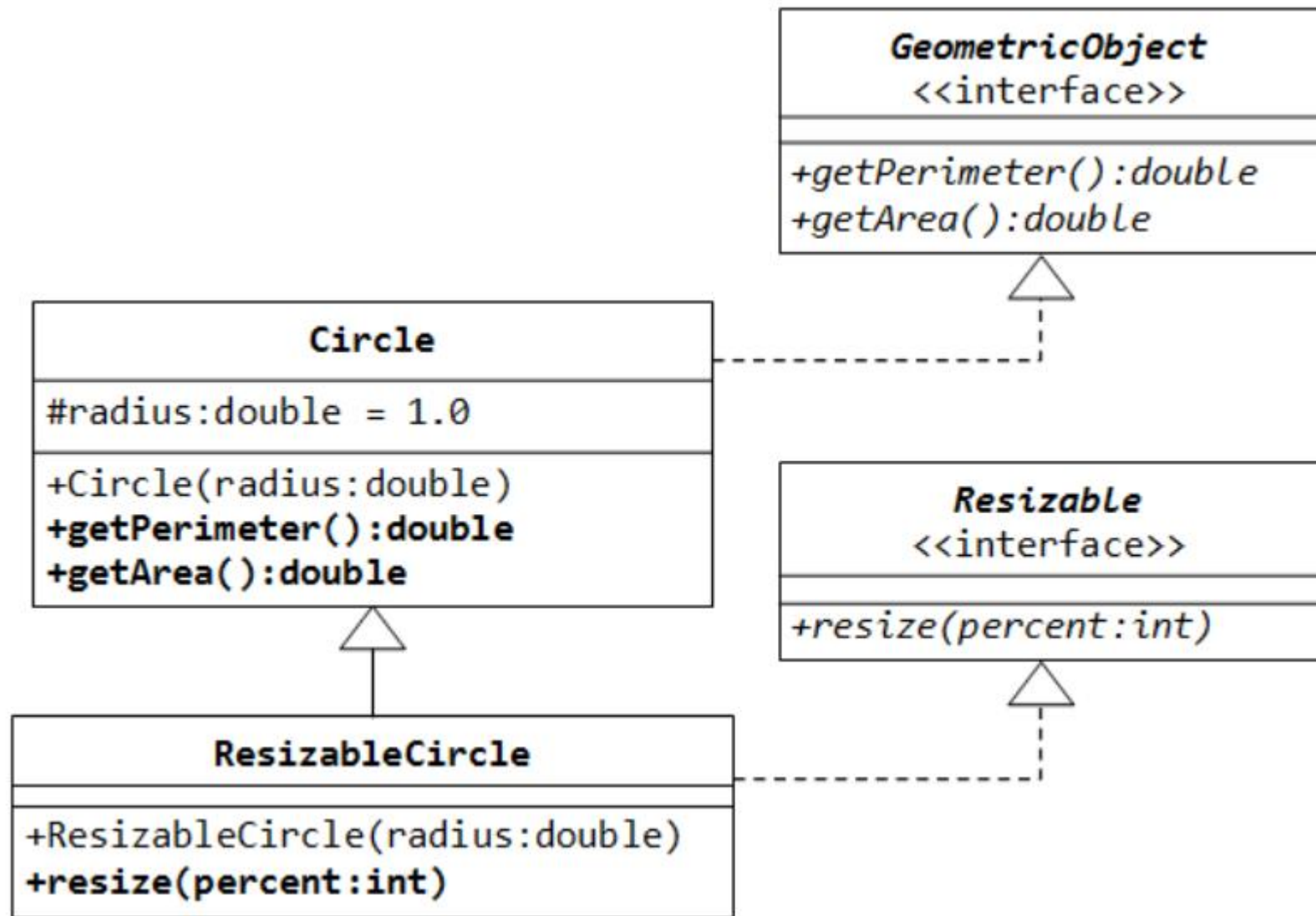
EXERCISES 06_01

Implement the following UML chart



EXERCISES 06_02

Implement the following UML chart



EXERCISES 06_03

- ❑ Write an application named `UseInsurance` that uses an abstract `Insurance` class and `Health` and `Life` subclasses to display different types of insurance policies and the cost per month.
- ❑ The `Insurance` class contains a `String` representing the type of insurance and a `double` that holds the monthly price. The `Insurance` class constructor requires a `String` argument indicating the type of insurance, but the `Life` and `Health` class constructors require no arguments.
- ❑ The `Insurance` class contains a `get` method for each field; it also contains two abstract methods named `setCost()` and `display()`.
- ❑ The `Life` class `setCost()` method sets the monthly fee to \$36, and the `Health` class sets the monthly fee to \$196. Write an application named `UseInsurance` that prompts the user for the type of insurance to be displayed, and then create the appropriate object. Save the files as **`Life.java`**, **`Health.java`**, **`Insurance.java`**, and **`UseInsurance.java`**.

EXERCISES 06_04

- ☐ Create an interface called `Runner`. The interface has an abstract method called `run()` that displays a message describing the meaning of *run* to the class.
- ☐ Create classes called `Machine`, `Athlete`, and `PoliticalCandidate` that all implement `Runner`.
- ☐ Create an application that demonstrates the use of the classes.
- ☐ Save the files as **`Runner.java`**, **`Machine.java`**, **`Athlete.java`**, **`PoliticalCandidate.java`**, and **`DemoRunners.java`**.

EXERCISES 06_05

- ❑ Create an interface named `Turner`, with a single method named `turn()`.
- ❑ Create a class named `Leaf` that implements `turn()` to display *Changing colors*.
- ❑ Create a class named `Page` that implements `turn()` to display *Going to the next page*. Create a class named `Pancake` that implements `turn()` to display *Flipping*.
- ❑ Write an application named `DemoTurners` that creates one object of each of these class types and demonstrates the `turn()` method for each class. Save the files as **`Turner.java`**, **`Leaf.java`**, **`Page.java`**, **`Pancake.java`**, and **`DemoTurners.java`**.
- ❑ Think of two more objects that use `turn()`, create classes for them, and then add objects to the `DemoTurners` application, renaming it **`DemoTurners2.java`**. Save the files, using the names of new objects that use `turn()`.

EXERCISES 06_06 PART 1

Sanchez Construction Loan Co. makes loans of up to \$100,000 for construction projects. There are two categories of Loans—those to businesses and those to individual applicants.

- ❑ Write an application that tracks all new construction loans. The application also must calculate the total amount owed at the due date (original loan amount + loan fee). The application should include the following classes:
- ❑ `Loan`—A public abstract class that implements the `LoanConstants` interface. A `Loan` includes a loan number, customer last name, amount of loan, interest rate, and term. The constructor requires data for each of the fields except interest rate. Do not allow loan amounts greater than \$100,000. Force any loan term that is not one of the three defined in the `LoanConstants` class to a short-term, 1-year loan. Create a `toString()` method that displays all the loan data.

EXERCISES 06_06 PART 2

- ❑ `LoanConstants`—A public interface class. `LoanConstants` includes constant values for short-term (1 year), medium-term (3 years), and long-term (5 years) loans. It also contains constants for the company name and the maximum loan amount.
- ❑ `BusinessLoan`—A public class that extends `Loan`. The `BusinessLoan` constructor sets the interest rate to 1% more than the current prime interest rate.
- ❑ `PersonalLoan`—A public class that extends `Loan`. The `PersonalLoan` constructor sets the interest rate to 2% more than the current prime interest rate.
- ❑ `CreateLoans`—An application that creates an array of five `Loans`. Prompt the user for the current prime interest rate. Then, in a loop, prompt the user for a loan type and all relevant information for that loan. Store the created `Loan` objects in the array. When data entry is complete, display all the loans.

Save the files as **`Loan.java`**, **`LoanConstants.java`**, **`BusinessLoan.java`**, **`PersonalLoan.java`**, and **`CreateLoans.java`**.

EXERCISES 06_07

- ❑ Declare the **PizzaDeluxe**, **PizzaSpecial** and **PizzaWoogy** classes. Each of these classes should extend the **Pizza** class. The **Pizza** class should implement the Comparable interface. The **Pizza** class should have the following private attributes:
 - calories:double
 - name:String
 - price:double
- ❑ The **Pizza** class should implements the Comparable interface and it should have the following public methods:
 - + Pizza(double, String, double)
 - + getCalories():double
 - + getName():String
 - + getPrice():double
 - + toString():java.util.String
 - + compareTo(Object):int
- ❑ In the classes that extend the Pizza class you should declare - in each one of them - at least three attributes, three methods and two constructors. Later, code a stand-alone application that instantiates the 3 concrete classes and creates an array (the array type is Pizza) of 40 different objects from the **PizzaDeluxe**, **PizzaSpecial** and **PizzaWoogy** classes. The stand-alone application should sort the 40 different objects according to their Price.

EXERCISES 06_08

- ❑ Create a `UsedCarException` class that extends `Exception`; its constructor receives a value for a vehicle identification number (VIN) that is passed to the parent constructor so it can be used in a `getMessage()` call.
- ❑ Save the class as **`UsedCarException.java`**. Create a `UsedCar` class with fields for VIN, make, year, mileage, and price.
- ❑ The `UsedCar` constructor throws a `UsedCarException` when the VIN is not four digits; when the make is not *Ford*, *Honda*, *Toyota*, *Chrysler*, or *Other*; when the year is not between 1997 and 2017 inclusive; or either the mileage or price is negative. Save the class as **`UsedCar.java`**.
- ❑ Write an application that establishes an array of at least seven `UsedCar` objects and handles any Exceptions. Display a list of only the `UsedCar` objects that were constructed successfully. Save the file as **`ThrowUsedCarException.java`**.